

Artificial Intelligence Laboratory

Viva Questions, Answers & Algorithm Reference

#	Topic
1	DFS & BFS – Graph Search Algorithms
2	A* Algorithm – Game Search Problem
3	N-Queens – Branch & Bound / Backtracking (CSP)
4	Rule-Based GUI Chatbot
5	Greedy Search – Selection Sort, MST, Shortest Path
6	Expert System – Medical Diagnosis

ASSIGNMENT 1 — Depth First Search (DFS) & Breadth First Search (BFS)

1.1 Algorithms

DFS — Recursive Algorithm

```
def dfs(graph, vertex, visited=None):
    if visited is None:
        visited = set()
    visited.add(vertex)
    print(vertex, end=' ') # Process node
    for neighbour in graph[vertex]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited)
    return visited

# Undirected graph as adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}
dfs(graph, 'A') # Output: A B D E F C
```

BFS — Iterative Algorithm (Queue)

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    while queue:
        vertex = queue.popleft()
        print(vertex, end=' ') # Process node
        for neighbour in graph[vertex]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

bfs(graph, 'A') # Output: A B C D E F
```

Complexity Comparison

Property	DFS	BFS
----------	-----	-----

Data Structure	Stack (Recursion)	Queue
Time Complexity	$O(V + E)$	$O(V + E)$
Space Complexity	$O(V)$	$O(V)$
Completeness	Not always	Yes (finite graph)
Optimality	No	Yes (unweighted)
Use Case	Topological sort	Shortest path

1.2 Viva Questions & Answers

Q: What is the main difference between DFS and BFS?

A: DFS explores as far as possible along each branch before backtracking (uses stack/recursion). BFS explores all neighbours at the current depth before going deeper (uses queue).

Q: What data structures are used in DFS and BFS?

A: DFS uses a Stack (or call stack via recursion). BFS uses a Queue (FIFO).

Q: Is DFS or BFS better for finding the shortest path?

A: BFS is better for unweighted graphs because it explores nodes level by level, guaranteeing the shortest path in terms of edges.

Q: What is the time complexity of DFS and BFS?

A: Both have time complexity $O(V + E)$, where V = vertices and E = edges.

Q: Can DFS get stuck in an infinite loop?

A: Yes, in graphs with cycles, DFS can loop infinitely if visited nodes are not tracked. A 'visited' set must be maintained.

Q: What is the space complexity of BFS?

A: $O(V)$ in the worst case, as all nodes at a level may be stored in the queue simultaneously.

Q: When would you prefer DFS over BFS?

A: DFS is preferred for: detecting cycles, topological sorting, solving mazes/puzzles, and when memory is constrained (BFS stores entire frontier).

Q: What is a recursive DFS and how does it work?

A: Recursive DFS uses the program's call stack instead of an explicit stack. It calls itself on each unvisited neighbour, naturally backtracking when a function returns.

Q: What is the difference between a tree and a graph for search?

A: A tree is an acyclic connected graph with a root. In graph search, we must track visited nodes to avoid cycles; in a tree, children are never revisited.

Q: Define connected graph and disconnected graph.

A: A connected graph has a path between every pair of vertices. In a disconnected graph, some vertices cannot be reached from others — DFS/BFS must be run from each unvisited component.

Q: What is the significance of the 'visited' array in graph traversal?

A: It prevents revisiting nodes, avoids infinite loops in cyclic graphs, and ensures each node is processed exactly once — maintaining $O(V+E)$ complexity.

Q: What is topological sorting and which algorithm is used?

A: Topological sorting linearly orders vertices of a Directed Acyclic Graph (DAG) such that for every edge $u \rightarrow v$, u comes before v . It is implemented using DFS with a finish-time stack.

ASSIGNMENT 2 — A* Algorithm for Game Search Problem

2.1 Concept

A* is an informed search algorithm that finds the shortest path from a start node to a goal. It uses the evaluation function: $f(n) = g(n) + h(n)$, where $g(n)$ is the cost from start to node n , and $h(n)$ is the heuristic (estimated cost from n to goal).

2.2 Algorithm (8-Puzzle / Grid Pathfinding)

```
import heapq

def heuristic(a, b):
    # Manhattan distance
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def a_star(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    open_set = []
    heapq.heappush(open_set, (0, start))
    came_from = {}
    g_score = {start: 0}
    f_score = {start: heuristic(start, goal)}

    while open_set:
        _, current = heapq.heappop(open_set)
        if current == goal:
            # Reconstruct path
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1]

        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = current[0]+dr, current[1]+dc
            neighbour = (nr, nc)
            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] != 1:
                tentative_g = g_score[current] + 1
                if tentative_g < g_score.get(neighbour, float('inf')):
                    came_from[neighbour] = current
                    g_score[neighbour] = tentative_g
                    f_score[neighbour] = tentative_g + heuristic(neighbour, goal)
```

```

heapq.heappush(open_set, (f_score[neighbour], neighbour))
return None # No path found

# 0=free, 1=wall
grid = [
[0,0,0,0,0],
[0,1,1,0,0],
[0,0,0,1,0],
[0,0,0,0,0],
]
path = a_star(grid, (0,0), (3,4))
print("Path:", path)

```

2.3 Viva Questions & Answers

Q: What does $f(n) = g(n) + h(n)$ mean in A*?

A: $f(n)$ is the total estimated cost. $g(n)$ is the actual cost from the start node to n . $h(n)$ is the heuristic (estimated cost from n to goal). A* always expands the node with the lowest $f(n)$.

Q: What is an admissible heuristic?

A: A heuristic $h(n)$ is admissible if it never overestimates the true cost to reach the goal: $h(n) \leq h^*(n)$. This guarantees A* finds the optimal solution.

Q: What is a consistent (monotone) heuristic?

A: $h(n)$ is consistent if for every node n and successor n' , $h(n) \leq \text{cost}(n, n') + h(n')$. Consistency implies admissibility and ensures nodes need not be re-expanded.

Q: Give examples of heuristics used in A*.

A: Manhattan distance (grid movement, no diagonals), Euclidean distance (continuous space), Misplaced tiles (8-puzzle), Hamming distance.

Q: How does A* differ from Dijkstra's algorithm?

A: Dijkstra uses $f(n) = g(n)$ only (no heuristic). A* adds $h(n)$ to guide the search toward the goal, making it faster in practice while still optimal when h is admissible.

Q: What is the time and space complexity of A*?

A: Both are $O(b^d)$ where b is branching factor and d is depth of solution. Space is the main limitation as A* stores all generated nodes in memory.

Q: When does A* become BFS?

A: When $h(n) = 0$ for all nodes. A* then degenerates to Dijkstra's/BFS since it uses only $g(n)$.

Q: What is the open list and closed list in A*?

A: Open list (priority queue): nodes discovered but not yet fully explored. Closed list (visited set): nodes already explored. A* expands the node with lowest $f(n)$ from the open list.

Q: What game problems can A* solve?

A: 8-puzzle, 15-puzzle, maze solving, grid-based pathfinding (games like chess, checkers), route planning in maps.

Q: What happens if the heuristic is inadmissible?

A: A* may not find the optimal path — it could return a suboptimal solution by overestimating and skipping the actual optimal path.

Q: What is Weighted A*?

A: Weighted A* uses $f(n) = g(n) + w \cdot h(n)$ where $w > 1$. It trades optimality for speed, finding solutions faster but possibly non-optimal — useful in real-time systems.

ASSIGNMENT 3 — CSP: N-Queens with Backtracking & Branch and Bound

3.1 Backtracking Solution — N-Queens

```
def solve_n_queens(n):
    solutions = []
    board = [-1] * n # board[i] = column of queen in row i

    def is_safe(row, col):
        for r in range(row):
            c = board[r]
            if c == col or abs(r - row) == abs(c - col):
                return False
        return True

    def backtrack(row):
        if row == n:
            solutions.append(board[:])
            return
        for col in range(n):
            if is_safe(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1 # Backtrack

    backtrack(0)
    return solutions

solutions = solve_n_queens(8)
print(f"Total solutions for 8-queens: {len(solutions)}") # 92
```

3.2 Branch and Bound Enhancement

```
def n_queens_bnb(n):
    # Using sets for O(1) constraint checking (Branch & Bound pruning)
    solutions = []
    cols = set()
    diag1 = set() # row - col
    diag2 = set() # row + col
    board = []

    def solve(row):
        if row == n:
```

```

solutions.append(board[:])
return
for col in range(n):
    if col in cols or (row-col) in diag1 or (row+col) in diag2:
        continue # Prune branch
    cols.add(col); diag1.add(row-col); diag2.add(row+col)
    board.append(col)
    solve(row + 1)
    board.pop()
    cols.discard(col); diag1.discard(row-col); diag2.discard(row+col)

solve(0)
return solutions

print(len(n_queens_bnb(8))) # 92 solutions

```

3.3 Graph Coloring with Backtracking

```

def graph_coloring(graph, num_colors):
    n = len(graph)
    color = [0] * n

    def is_safe(node, c):
        for neighbour in graph[node]:
            if color[neighbour] == c:
                return False
        return True

    def solve(node):
        if node == n:
            return True
        for c in range(1, num_colors + 1):
            if is_safe(node, c):
                color[node] = c
                if solve(node + 1):
                    return True
        color[node] = 0 # Backtrack
        return False

    # Adjacency list
    graph_ex = [[1,2,3], [0,2], [0,1,3], [0,2]]
    color = [0] * len(graph_ex)
    solve(0)
    print("Colors assigned:", color)

```

3.4 Viva Questions & Answers

Q: What is a Constraint Satisfaction Problem (CSP)?

A: A CSP is defined by variables, domains (possible values), and constraints. The goal is to assign values to variables from their domains such that all constraints are satisfied. Examples: N-Queens, Sudoku, Graph Coloring.

Q: Explain backtracking in the context of N-Queens.

A: Backtracking places a queen row by row. If placing in a column is safe (no conflicts), it recurses to the next row. If no column is safe, it removes the queen (backtracks) and tries the next column in the previous row.

Q: What is the difference between backtracking and brute force?

A: Brute force tries all possible combinations. Backtracking prunes invalid branches early — as soon as a partial assignment violates a constraint, it abandons that branch instead of completing it.

Q: What are the three types of constraints in CSP?

A: Unary constraints (restrict a single variable), Binary constraints (between two variables, e.g., queens can't be in same row/column), and Global constraints (involve more than two variables, e.g., alldiff).

Q: How many solutions exist for the 8-Queens problem?

A: 92 distinct solutions. If reflections and rotations are considered the same, there are 12 fundamental solutions.

Q: What is Branch and Bound?

A: Branch and Bound is a systematic enumeration technique. It partitions the solution space into branches and uses a bound (lower/upper estimate) to prune branches that cannot lead to a better solution than the current best.

Q: What is Forward Checking in CSP?

A: Forward Checking, after assigning a value to a variable, removes inconsistent values from the domains of unassigned variables. If any domain becomes empty, it backtracks immediately.

Q: What is the Minimum Remaining Values (MRV) heuristic?

A: MRV selects the variable with the fewest legal values remaining in its domain ('fail-first' strategy). This detects failures earlier and reduces backtracking.

Q: What is constraint propagation?

A: Constraint propagation enforces constraints locally to reduce domains before or during search. Arc Consistency (AC-3) is a common algorithm that removes values that have no valid counterpart in neighboring variables.

Q: What is the chromatic number of a graph?

A: The minimum number of colors needed to color a graph such that no two adjacent vertices share the same color. Finding it is NP-hard in general.

Q: Explain arc consistency (AC-3).

A: AC-3 ensures that for every arc (X_i, X_j) , every value in $\text{domain}(X_i)$ has at least one consistent value in $\text{domain}(X_j)$. If a value is removed from $\text{domain}(X_i)$, all arcs pointing to X_i are re-checked.

ASSIGNMENT 4 — Rule-Based GUI Chatbot

4.1 Rule-Based Engine

```
import re

# Knowledge base: pattern -> response
RULES = [
    (r'hello|hi|hey', "Hello! Welcome to TechSupport. How can I help you today?"),
    (r'order.*status|track', "Please share your order ID and I'll check the status for you."),
    (r'refund|return', "Refunds are processed within 5-7 business days. Shall I initiate one?"),
    (r'password.*reset', "Click 'Forgot Password' on the login page to reset your password."),
    (r'speak.*agent|human', "Connecting you to a live agent. Please wait..."),
    (r'hours|timing', "Our support hours are Mon-Sat, 9 AM to 6 PM IST."),
    (r'thank|bye|goodbye', "Thank you for contacting us! Have a great day!"),
]

def get_response(user_input):
    user_input = user_input.lower().strip()
    for pattern, response in RULES:
        if re.search(pattern, user_input):
            return response
    return "I'm sorry, I didn't understand that. Could you rephrase or type 'agent' for help?"

# GUI using tkinter
import tkinter as tk

def send_message():
    msg = entry.get().strip()
    if not msg: return
    chat.config(state='normal')
    chat.insert('end', f"You: {msg}"
    ", 'user')
    reply = get_response(msg)
    chat.insert('end', f"Bot: {reply}"
    ", 'bot')
    chat.config(state='disabled')
    chat.see('end')
    entry.delete(0, 'end')
```

```

root = tk.Tk()
root.title("Customer Support Chatbot")
chat = tk.Text(root, state='disabled', width=60, height=20,
font=('Arial', 11), bg='#f0f4f8')
chat.tag_config('user', foreground='#1565c0', font=('Arial', 11, 'bold'))
chat.tag_config('bot', foreground='#2e7d32')
chat.pack(padx=10, pady=10)
entry = tk.Entry(root, width=50, font=('Arial', 11))
entry.pack(side='left', padx=10, pady=5)
btn = tk.Button(root, text='Send', command=send_message, bg='#1565c0',
fg='white', font=('Arial', 11, 'bold'))
btn.pack(side='left')
root.mainloop()

```

4.2 Viva Questions & Answers

Q: What is a rule-based chatbot?

A: A rule-based chatbot uses a predefined set of if-then rules and pattern matching (often regex) to produce responses. It does not learn — responses are hard-coded by developers.

Q: What are the limitations of a rule-based chatbot?

A: Cannot handle unseen inputs, lacks contextual understanding, needs manual rule creation for every scenario, doesn't improve with usage, and can't handle ambiguous or complex queries.

Q: What is the difference between a rule-based and AI-based chatbot?

A: Rule-based chatbots match patterns to fixed responses. AI-based chatbots use NLP and ML to understand intent, context, and generate dynamic responses (e.g., ChatGPT, Dialogflow).

Q: What is intent recognition?

A: Intent recognition identifies the purpose behind a user's message. E.g., 'What is my order status?' maps to the intent 'track_order'. Used in NLP-based chatbots.

Q: What is entity extraction?

A: Entity extraction pulls specific information from user input (e.g., order ID, dates, product names). Combined with intent, it enables precise responses.

Q: What is NLP and how does it apply to chatbots?

A: Natural Language Processing (NLP) enables computers to understand and generate human language. In chatbots, NLP handles tokenization, POS tagging, intent classification, and named entity recognition.

Q: What is the Turing Test?

A: A test proposed by Alan Turing in 1950: if a human evaluator cannot distinguish between a machine and a human through text conversation, the machine is considered intelligent.

Q: What is the role of regex in rule-based chatbots?

A: Regular expressions match user input patterns. E.g., `r'order.*status'` matches 'order status', 'check my order status', 'I want to know order status' — all with one rule.

Q: What is sentiment analysis in chatbots?

A: Sentiment analysis detects the emotional tone of a user message (positive, negative, neutral). Chatbots use it to escalate to a human agent when the user is frustrated.

Q: What are slots in chatbot design?

A: Slots are variables that store information collected during conversation (e.g., name, date, issue type). Slot-filling guides the chatbot to gather all required information before responding.

Q: Name some chatbot frameworks or platforms.

A: Rasa (open-source NLP), Dialogflow (Google), Microsoft Bot Framework, Amazon Lex, IBM Watson Assistant, ChatterBot (Python).

ASSIGNMENT 5 — Greedy Search Algorithms

5.1 Selection Sort

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n): # Find minimum in remaining
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i] # Greedy swap
    return arr

arr = [64, 25, 12, 22, 11]
print(selection_sort(arr)) # [11, 12, 22, 25, 64]
# Time: O(n^2), Space: O(1), Unstable sort
```

5.2 Kruskal's Minimum Spanning Tree (Greedy)

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n
    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
    def union(self, x, y):
        rx, ry = self.find(x), self.find(y)
        if rx == ry: return False
        if self.rank[rx] < self.rank[ry]: rx, ry = ry, rx
        self.parent[ry] = rx
        if self.rank[rx] == self.rank[ry]: self.rank[rx] += 1
        return True

def kruskal(n, edges):
    edges.sort(key=lambda e: e[2]) # Greedy: sort by weight
    uf = UnionFind(n)
    mst = []
    for u, v, w in edges:
        if uf.union(u, v): # Add if no cycle
            mst.append((u, v, w))
    if len(mst) == n - 1: break
```

```

return mst

edges = [(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]
print(kruskal(4, edges)) # [(2,3,4),(0,3,5),(0,1,10)] weight=19

```

5.3 Dijkstra's Shortest Path (Greedy)

```

import heapq

def dijkstra(graph, src):
    dist = {node: float('inf') for node in graph}
    dist[src] = 0
    pq = [(0, src)] # (distance, node)

    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]: continue # Stale entry
        for v, w in graph[u]:
            if dist[u] + w < dist[v]: # Greedy relaxation
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist

graph = {
    'A': [('B',4),('C',1)],
    'B': [('D',1)],
    'C': [('B',2),('D',5)],
    'D': []
}
print(dijkstra(graph, 'A'))
# {'A':0, 'B':3, 'C':1, 'D':4}

```

5.4 Prim's MST Algorithm

```

import heapq

def prim(graph, start):
    visited = set([start])
    edges = [(w, start, v) for v, w in graph[start]]
    heapq.heapify(edges)
    mst = []

    while edges:
        w, u, v = heapq.heappop(edges)
        if v in visited: continue # Skip already visited
        visited.add(v)
        mst.append((u, v, w))

```

```

for next_v, next_w in graph[v]:
    if next_v not in visited:
        heapq.heappush(edges, (next_w, v, next_v))
return mst

graph = {
0: [(1,2),(3,6)], 1: [(0,2),(2,3),(3,8),(4,5)],
2: [(1,3),(4,7)], 3: [(0,6),(1,8)], 4: [(1,5),(2,7)]
}

print(prim(graph, 0))

```

5.5 Viva Questions & Answers

Q: What is the Greedy approach?

A: Greedy algorithms make the locally optimal choice at each step, hoping to find the global optimum. They never reconsider past choices. They work when optimal substructure and the greedy choice property hold.

Q: Does greedy always give the optimal solution?

A: No. Greedy works for MST (Kruskal, Prim), Dijkstra, Activity Selection, Huffman Coding, but fails for 0/1 Knapsack, TSP, and many NP-hard problems.

Q: What is a Minimum Spanning Tree (MST)?

A: An MST of a connected weighted graph is a spanning tree with the minimum total edge weight. A spanning tree has $V-1$ edges and connects all V vertices without cycles.

Q: What is the difference between Prim's and Kruskal's algorithm?

A: Prim's grows the MST from a starting vertex by adding the cheapest edge connecting the tree to a non-tree vertex. Kruskal's sorts all edges and adds them one by one, skipping edges that form cycles.

Q: Which is better: Prim's or Kruskal's?

A: Prim's is better for dense graphs (many edges). Kruskal's is better for sparse graphs. Prim's with a min-heap: $O(E \log V)$. Kruskal's: $O(E \log E)$.

Q: What data structure does Dijkstra use and why?

A: A min-priority queue (binary heap or Fibonacci heap) to always extract the vertex with the smallest known distance, enabling the greedy relaxation step efficiently.

Q: What is the limitation of Dijkstra's algorithm?

A: Dijkstra cannot handle negative edge weights. For graphs with negative weights, Bellman-Ford should be used instead.

Q: What is the greedy choice property?

A: A problem has the greedy choice property if a globally optimal solution can be reached by making a locally optimal (greedy) choice at each step — i.e., local choices lead to global optimum.

Q: What is Selection Sort's time complexity and is it stable?

A: Time complexity is $O(n^2)$ for all cases. Selection Sort is NOT stable — it may change the relative order of equal elements. Space complexity is $O(1)$.

Q: What is a Union-Find data structure used in Kruskal's?

A: Union-Find (Disjoint Set Union) tracks which vertices are in the same connected component. It efficiently detects if adding an edge creates a cycle using `union()` and `find()` operations with path compression.

Q: What is the Job Scheduling Problem and how does Greedy solve it?

A: Given jobs with deadlines and profits, maximize total profit by scheduling jobs before their deadlines. Greedy approach: sort by profit descending, schedule each job in the latest available slot before its deadline.

Q: What is Huffman Coding and where does Greedy apply?

A: Huffman Coding is a greedy compression algorithm. It assigns shorter binary codes to more frequent characters using a min-heap to always merge the two lowest-frequency nodes first.

ASSIGNMENT 6 — Expert System: Medical Diagnosis

6.1 Architecture Overview

An Expert System has three components: (1) Knowledge Base — stores domain facts and rules, (2) Inference Engine — applies rules using forward/backward chaining, (3) User Interface — collects symptoms and presents diagnosis.

6.2 Python Implementation — Medical Expert System

[illegible]

[illegible]

6.3 Viva Questions & Answers

Q: What is an Expert System?

A: An Expert System is an AI program that mimics the decision-making ability of a human expert in a specific domain. It uses a knowledge base of rules and an inference engine to solve complex problems.

Q: What are the main components of an Expert System?

A: 1) Knowledge Base (facts + rules), 2) Inference Engine (reasoning mechanism), 3) Working Memory (current facts/session), 4) User Interface, 5) Explanation Facility (justifies conclusions).

Q: What is forward chaining?

A: Forward chaining starts from known facts and applies rules to infer new facts until the goal is reached. It is data-driven. Used in: medical diagnosis, business rule engines.

Q: What is backward chaining?

A: Backward chaining starts from the goal and works backward to find supporting facts. It is goal-driven. Used in: theorem proving, Prolog programming.

Q: What is the difference between forward and backward chaining?

A: Forward chaining: facts $\rightarrow$ rules $\rightarrow$ conclusion (data-driven). Backward chaining: goal $\leftarrow$ rules $\leftarrow$ facts (goal-driven). Forward chaining explores all possibilities; backward is more focused.

Q: What is a knowledge base?

A: A structured repository of domain-specific facts (assertions) and rules (if-then statements) representing the knowledge of a human expert. It is the heart of an Expert System.

Q: What is knowledge engineering?

A: The process of acquiring, representing, and organizing domain knowledge from human experts into a computer-readable form. The person doing this is called a knowledge engineer.

Q: What are the limitations of Expert Systems?

A: Knowledge acquisition bottleneck, brittleness (fails outside defined domain), inability to learn from experience, cannot handle uncertainty well, expensive to build and maintain.

Q: What is a fuzzy expert system?

A: A fuzzy expert system handles uncertainty using fuzzy logic instead of crisp true/false values. Rules use linguistic variables (e.g., 'temperature is HIGH') with degrees of membership.

Q: What is an inference engine?

A: The processing unit of an expert system that applies rules from the knowledge base to the working memory to derive new conclusions or recommendations. It can use forward or backward chaining.

Q: Name real-world applications of expert systems.

A: MYCIN (bacterial infection diagnosis), DENDRAL (chemical structure analysis), R1/XCON (DEC computer configuration), credit approval systems, fraud detection, airline scheduling.

Q: What is the Explanation Facility in an Expert System?

A: A component that explains HOW (which rules were used) and WHY (why a question was asked) a conclusion was reached. This transparency builds user trust in the system's recommendations.